UTS

PEMBELAJARAN MESIN



NAMA : Paris Karina Naswa

NIM : 202431140

KELAS : G

DOSEN : Dr. Dra. Dwina Kuswardani, M.Kom

NO.PC : -

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Morgan Immanuel Nainggolan

3.

4.

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI *2*
BAB I *3*
PENDAHULUAN *3*
1.1 Rumusan Masalah **3**
1.2 Tujuan Masalah **3**
1.3 Manfaat Masalah **3**
BAB II *4*
LANDASAN TEORI *4*
2.1 Machine Learning **4**
2.2 Exploratory Data Analysis (EDA) **4**
2.3 Preprocessing Data **4**
2.4 Algoritma Decision Tree **6**
2.5 Algoritma K-Nearest Neighbors (KNN) **7**
2.6 Evaluasi Performa Model Klasifikasi **8**
2.7 Perbandingan Algoritma Decision Tree dan KNN **9**
BAB III *11*
HASIL *11*
3.1 Import **11**
3.2 Analisis EDA **12**
3.3 Decision Tree **13**
3.4 KNN **17**
BAB IV *20*
PENUTUP *20*
DAFTAR PUSTAKA *21*

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

1.1.1 Bagaimana karakteristik data awal melalui tahapan Exploratory Data Analysis (EDA) serta pengaruh tahapan pra-pemrosesan (label encoding dan feature scaling) terhadap kesiapan dataset?

1.1.2 Bagaimana performa dan hasil evaluasi model klasifikasi Decision Tree dalam memprediksi pemenang ronde (round winner) berdasarkan struktur pohon keputusan yang terbentuk?

1.1.3 Bagaimana pengaruh variasi nilai K terhadap tingkat akurasi pada algoritma K-Nearest Neighbors (KNN) dengan penskalaan fitur?

1.2 Tujuan Masalah

1.2.1 Mengetahui karakteristik struktur data awal melalui tahapan Exploratory Data Analysis (EDA) serta memahami efektivitas tahapan pra-pemrosesan data (label encoding dan feature scaling) dalam mempersiapkan dataset agar optimal digunakan dalam pelatihan model machine learning.

1.2.2 Mengevaluasi kinerja, tingkat akurasi, serta kemampuan generalisasi dari model klasifikasi Decision Tree dalam memprediksi pemenang ronde (round winner) berdasarkan pembentukan struktur pohon keputusan dan matriks evaluasi kinerja.

1.2.3 Menganalisis pengaruh variasi nilai K terhadap tingkat akurasi model K-Nearest Neighbors (KNN) yang telah melalui proses standarisasi fitur (StandardScaler), guna menentukan nilai hyperparameter K yang paling optimal dalam meminimalkan kesalahan klasifikasi.

1.3 Manfaat Masalah

1.3.1 Memahami Urgensi Tahapan Pra-Pemrosesan Data dalam Pembelajaran Mesin

1.3.2 Meningkatkan Kemampuan Analisis dan Evaluasi Performa Algoritma Klasifikasi

1.3.3 Menguasai Teknik Optimasi Hyperparameter untuk Model Berbasis Jarak

BAB II
LANDASAN TEORI

2.1 Machine Learning

Machine Learning merupakan salah satu cabang kecerdasan buatan (Artificial Intelligence) yang memungkinkan komputer mempelajari pola dan hubungan dari sekumpulan data untuk menghasilkan prediksi atau keputusan tanpa harus diprogram secara eksplisit untuk setiap kondisi. Dalam penerapannya, Machine Learning memanfaatkan data historis sebagai dasar pembentukan model yang selanjutnya dapat digunakan untuk mengklasifikasikan data baru. Kumar dan Bhardwaj [1] menjelaskan bahwa penerapan algoritma Machine Learning dapat digunakan untuk menemukan pola tersembunyi dalam dataset serta membangun model prediktif melalui proses pengolahan dan evaluasi data. Dalam penelitian ini, Machine Learning digunakan untuk mempelajari hubungan antara atribut-atribut dalam pertandingan Counter-Strike 2 dengan hasil akhir ronde, yaitu pihak yang memenangkan ronde (round winner).

2.2 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) merupakan tahapan awal dalam pengolahan data yang bertujuan untuk memahami karakteristik, struktur, distribusi, serta hubungan antarvariabel dalam suatu dataset sebelum diterapkan algoritma Machine Learning. Proses EDA dilakukan melalui pemeriksaan informasi dataset, identifikasi tipe data, pengecekan nilai kosong (missing values), analisis statistik deskriptif, serta visualisasi data menggunakan histogram, diagram batang, dan heatmap korelasi. Kumar dan Bhardwaj [1] menerapkan analisis variabel numerik, analisis variabel kategorikal, korelasi numerik, serta analisis bivariat sebagai bagian dari tahapan eksplorasi dan persiapan data. Oleh karena itu, EDA memiliki peran penting dalam mengidentifikasi karakteristik dataset Counter-Strike 2, termasuk distribusi kelas round winner, persebaran nilai setiap fitur, serta kemungkinan adanya ketidakseimbangan data yang dapat memengaruhi proses klasifikasi.

2.3 Preprocessing Data

Preprocessing data atau pra-pemrosesan data merupakan proses mempersiapkan dataset agar dapat digunakan secara optimal dalam pembentukan model Machine Learning. Tahapan ini diperlukan karena data mentah sering kali memiliki karakteristik yang belum sesuai dengan kebutuhan algoritma, seperti perbedaan tipe data, nilai yang tidak lengkap, variabel kategorikal, dan perbedaan rentang nilai antarfitur. Kumar dan Bhardwaj [1] menunjukkan bahwa penerapan teknik encoding dan scaling merupakan bagian penting dalam mempersiapkan data sebelum proses pelatihan model klasifikasi. Penelitian tersebut juga mengevaluasi pengaruh teknik pra-pemrosesan terhadap performa beberapa algoritma, termasuk Decision Tree dan K-Nearest Neighbors. Dalam praktikum ini, preprocessing difokuskan pada penerapan Label Encoding untuk

mengubah data kategorikal menjadi numerik dan StandardScaler untuk menyamakan skala fitur numerik, khususnya dalam penerapan algoritma KNN.

2.3.1 Label Encoding

Label Encoding merupakan teknik pra-pemrosesan yang digunakan untuk mengubah nilai kategorikal atau berbentuk teks menjadi representasi numerik berupa bilangan bulat. Teknik ini memungkinkan atribut yang sebelumnya berbentuk kategori dapat diproses oleh algoritma Machine Learning yang membutuhkan masukan numerik. Kumar dan Bhardwaj [1] membahas penggunaan teknik encoding sebagai bagian dari proses persiapan dataset untuk meningkatkan kesiapan data dalam penerapan berbagai algoritma klasifikasi. Pada dataset Counter-Strike 2, Label Encoding dapat diterapkan terhadap atribut kategorikal, seperti nama senjata, jenis perlengkapan, atau kategori lainnya, sesuai dengan struktur data yang tersedia. Melalui proses tersebut, setiap kategori akan direpresentasikan menggunakan nilai numerik tertentu sehingga dapat digunakan dalam proses pelatihan model. Namun, pemberian angka pada kategori tidak selalu menunjukkan adanya tingkatan atau hubungan urutan antar kategori, sehingga hasil encoding perlu diinterpretasikan sesuai karakteristik atributnya.

2.3.2 Feature Scaling

Feature Scaling merupakan proses penyamaan rentang atau skala nilai antarfitur numerik agar perbedaan besaran nilai tidak menyebabkan suatu fitur mendominasi proses perhitungan algoritma. Tahapan ini sangat penting bagi algoritma yang menggunakan perhitungan jarak, salah satunya K-Nearest Neighbors. Manurung et al. [3] menjelaskan bahwa performa algoritma KNN sangat bergantung pada penerapan feature scaling karena perbedaan skala antarfitur dapat memengaruhi hasil pengukuran jarak dan menurunkan akurasi klasifikasi. Oleh sebab itu, penskalaan fitur diperlukan agar setiap atribut numerik memiliki kontribusi yang lebih seimbang dalam proses perhitungan jarak. Dalam praktikum ini, teknik yang digunakan adalah StandardScaler untuk melakukan standarisasi fitur numerik sebelum proses pelatihan model KNN.

2.3.3 StandardScaler

StandardScaler merupakan salah satu teknik standarisasi data yang mengubah nilai setiap fitur numerik berdasarkan nilai rata-rata (mean) dan standar deviasi (standard deviation) sehingga menghasilkan data dengan rata-rata mendekati nol dan standar deviasi mendekati satu.

Manurung et al. [3] menjelaskan bahwa penerapan feature scaling pada KNN diperlukan untuk mengurangi pengaruh perbedaan skala atribut terhadap perhitungan jarak. Dengan demikian, StandardScaler digunakan dalam praktikum ini untuk mempersiapkan fitur numerik agar memiliki skala yang sebanding sehingga proses penentuan tetangga terdekat pada algoritma KNN dapat berjalan lebih proporsional.

2.4 Algoritma Decision Tree

Decision Tree merupakan algoritma Machine Learning berbasis pembelajaran terawasi (supervised learning) yang digunakan untuk menyelesaikan permasalahan klasifikasi maupun regresi. Algoritma ini membentuk struktur menyerupai pohon yang terdiri atas root node, internal node, branch, dan leaf node. Setiap node merepresentasikan proses pengambilan keputusan berdasarkan kondisi atau karakteristik tertentu dari fitur yang digunakan. Kumar dan Bhardwaj [1] menjelaskan bahwa pembentukan Decision Tree dapat menggunakan beberapa metode, seperti ID3, CART, dan C4.5, yang bekerja dengan membagi data berdasarkan kriteria tertentu untuk menghasilkan keputusan. Sementara itu, Sasmita et al. [2] menerapkan Decision Tree untuk mengklasifikasikan kategori kualitas udara dan mengevaluasi hasilnya menggunakan sejumlah metrik performa. Dalam praktikum ini, Decision Tree digunakan untuk memprediksi pihak yang memenangkan ronde (round winner) berdasarkan karakteristik dan kondisi yang terdapat pada data pertandingan Counter-Strike 2.

2.4.1 Struktur Pohon Keputusan

Struktur Decision Tree terdiri atas beberapa komponen utama yang memiliki fungsi berbeda dalam proses pengambilan keputusan. Root node merupakan simpul awal yang merepresentasikan keseluruhan dataset, sedangkan internal node merupakan simpul yang digunakan untuk menguji kondisi berdasarkan atribut tertentu. Setiap hasil pengujian akan membentuk branch atau cabang yang mengarahkan proses menuju simpul berikutnya. Adapun leaf node merupakan simpul akhir yang menunjukkan hasil prediksi atau kelas yang dihasilkan oleh model. Dalam proses pembentukannya, Decision Tree melakukan pemisahan data berdasarkan atribut yang dianggap paling baik dalam membedakan kelas. Kumar dan Bhardwaj [1] menjelaskan bahwa algoritma pembentukan pohon keputusan dapat menggunakan pendekatan seperti CART dan C4.5 untuk menentukan pemisahan data berdasarkan kriteria tertentu. Struktur tersebut memungkinkan proses klasifikasi dapat ditelusuri melalui aturan keputusan yang terbentuk pada setiap cabang pohon.

2.4.2 Gini Impurity

Gini Impurity merupakan salah satu metode yang digunakan untuk mengukur tingkat ketidakmurnian (impurity) suatu node dalam algoritma Decision Tree. Nilai Gini digunakan untuk mengetahui seberapa baik suatu atribut dalam memisahkan data ke dalam kelas yang berbeda. Semakin kecil nilai Gini Impurity, semakin tinggi tingkat kemurnian kelas dalam node tersebut.

Kumar dan Bhardwaj [1] menjelaskan penggunaan Gini Index sebagai salah satu kriteria dalam proses pembentukan Decision Tree. Melalui perhitungan tersebut, algoritma dapat menentukan atribut yang menghasilkan pemisahan data paling baik berdasarkan tingkat kemurnian kelas. Dalam klasifikasi round winner, kriteria ini dapat membantu model menentukan kombinasi atribut yang mampu membedakan kondisi kemenangan antara pihak Terrorist dan Counter-Terrorist.

2.4.3 Evaluasi Model Decision Tree

Evaluasi model merupakan tahapan yang dilakukan untuk mengetahui kemampuan algoritma dalam menghasilkan prediksi yang sesuai dengan data aktual. Salah satu metode yang digunakan adalah Confusion Matrix, yaitu tabel yang membandingkan hasil prediksi model dengan kelas sebenarnya. Sasmita et al. [2] menggunakan metrik accuracy, precision, recall, dan F1-score untuk mengevaluasi performa algoritma Decision Tree dan KNN dalam klasifikasi kualitas udara. Hasil penelitian tersebut menunjukkan bahwa Decision Tree memperoleh akurasi sebesar 90,67%, precision 90,61%, recall 90,67%, dan F1-score 90,63%. Penggunaan beberapa metrik evaluasi tersebut memberikan gambaran yang lebih menyeluruh dibandingkan hanya mengandalkan nilai akurasi. Dalam praktikum ini, evaluasi dilakukan untuk mengetahui kemampuan Decision Tree dalam memprediksi round winner serta mengidentifikasi kesalahan klasifikasi yang dihasilkan model.

2.5 Algoritma K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) merupakan algoritma klasifikasi dalam pembelajaran terawasi (supervised learning) yang menentukan kelas suatu data berdasarkan kedekatannya dengan sejumlah data lain yang telah memiliki label. Algoritma ini bekerja dengan menghitung jarak antara data yang akan diprediksi dengan seluruh data pada data pelatihan, kemudian memilih sejumlah tetangga terdekat sebanyak nilai K yang telah ditentukan. Kelas dengan jumlah kemunculan terbanyak di antara tetangga tersebut akan menjadi hasil prediksi. Kumar dan Bhardwaj [1] menjelaskan bahwa KNN merupakan salah satu algoritma klasifikasi yang digunakan dalam pemodelan prediktif dan dapat diterapkan untuk mengidentifikasi pola berdasarkan kedekatan karakteristik data. Selain itu, Manurung et al. [3] menjelaskan bahwa performa KNN sangat dipengaruhi oleh pemilihan nilai K dan penerapan feature scaling. Dalam praktikum ini, KNN digunakan untuk memprediksi round winner berdasarkan kemiripan karakteristik kondisi pertandingan Counter-Strike 2.

2.5.1 Perhitungan Euclidean Distance

Euclidean Distance merupakan metode pengukuran jarak yang umum digunakan dalam algoritma KNN untuk mengetahui tingkat kedekatan antara dua titik data. Perhitungan dilakukan dengan mencari akar kuadrat dari jumlah selisih kuadrat setiap atribut antara data yang akan diprediksi dan data pelatihan. Semakin kecil nilai jarak yang diperoleh, semakin dekat kemiripan karakteristik kedua data tersebut. Rumus Euclidean Distance adalah sebagai berikut:

$$d(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Keterangan:
- (d(x,y)) = Jarak antara dua titik data.
- (x_i) = Nilai fitur pada data pertama.

- (y_i) = Nilai fitur pada data kedua.
- (n) = Jumlah fitur yang digunakan.

Kumar dan Bhardwaj [1] menjelaskan penggunaan pengukuran Euclidean Distance dalam menentukan kedekatan antarobjek pada algoritma KNN. Perhitungan ini menjadi dasar bagi KNN dalam menentukan sejumlah tetangga terdekat yang akan digunakan untuk proses klasifikasi. Dengan demikian, data pertandingan yang memiliki karakteristik paling mendekati data baru akan menjadi dasar penentuan prediksi kelas round winner.

2.5.2 Pengaruh Variasi Nilai K

Nilai K merupakan parameter utama (hyperparameter) dalam algoritma KNN yang menentukan jumlah tetangga terdekat yang digunakan untuk melakukan klasifikasi. Pemilihan nilai K dapat memengaruhi hasil prediksi dan tingkat akurasi model. Nilai K yang terlalu kecil dapat menyebabkan model lebih sensitif terhadap noise atau data yang menyimpang, sedangkan nilai K yang terlalu besar dapat menyebabkan model kurang mampu menangkap pola lokal karena mempertimbangkan terlalu banyak tetangga. Manurung et al. [3] menjelaskan bahwa pemilihan nilai K yang kurang tepat dapat menurunkan performa klasifikasi sehingga diperlukan proses optimasi parameter untuk memperoleh konfigurasi yang lebih baik. Penelitian tersebut menerapkan Grid Search Cross-Validation untuk menguji kombinasi parameter dan memperoleh konfigurasi KNN yang optimal. Oleh karena itu, praktikum ini melakukan pengujian terhadap beberapa variasi nilai K untuk mengetahui pengaruhnya terhadap akurasi klasifikasi serta menentukan nilai K yang menghasilkan performa terbaik pada dataset Counter-Strike 2.

2.6 Evaluasi Performa Model Klasifikasi

2.6.1 Confusion Matrix

Confusion Matrix merupakan tabel evaluasi yang digunakan untuk membandingkan hasil prediksi model dengan kelas aktual. Pada klasifikasi dua kelas, Confusion Matrix terdiri atas empat komponen utama, yaitu:

Komponen	Keterangan
True Positive (TP)	Data positif yang diprediksi positif dengan benar.
True Negative (TN)	Data negatif yang diprediksi negatif dengan benar.
False Positive (FP)	Data negatif yang salah diprediksi sebagai positif.
False Negative (FN)	Data positif yang salah diprediksi sebagai negatif.

Dalam penelitian ini, Confusion Matrix digunakan untuk mengidentifikasi jumlah prediksi yang benar maupun salah dalam menentukan pihak pemenang ronde. Hasil dari tabel tersebut selanjutnya digunakan untuk menghitung berbagai metrik evaluasi model.

2.6.2 Accuracy

Accuracy merupakan metrik yang digunakan untuk mengukur persentase keseluruhan prediksi yang benar dibandingkan dengan seluruh data pengujian. Nilai accuracy yang semakin tinggi menunjukkan bahwa model memiliki tingkat ketepatan prediksi yang semakin baik.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%$$

2.6.3 Precision

Precision merupakan metrik yang digunakan untuk mengukur ketepatan model ketika memprediksi suatu kelas positif. Metrik ini menunjukkan seberapa banyak prediksi positif yang benar dibandingkan dengan seluruh data yang diprediksi sebagai positif.

$$Precision = \frac{TP}{TP + FP} \times 100\%$$

2.6.4 Recall

Recall merupakan metrik yang digunakan untuk mengukur kemampuan model dalam menemukan seluruh data yang sebenarnya termasuk dalam kelas positif. Nilai recall yang tinggi menunjukkan bahwa model mampu mengenali sebagian besar data positif dengan benar.

$$Recall = \frac{TP}{TP + FN} \times 100\%$$

2.6.5 F1-Score

F1-Score merupakan metrik evaluasi yang menggabungkan precision dan recall menggunakan rata-rata harmonik. Metrik ini berguna untuk memberikan gambaran keseimbangan antara ketepatan prediksi positif dan kemampuan model dalam menemukan data positif.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Keempat metrik tersebut dapat digunakan secara bersamaan untuk memperoleh gambaran performa model yang lebih komprehensif. Hal ini sejalan dengan penelitian Sasmita et al. [2] yang tidak hanya menggunakan accuracy, tetapi juga precision, recall, dan F1-score dalam mengevaluasi algoritma klasifikasi.

2.7 Perbandingan Algoritma Decision Tree dan KNN

Decision Tree dan KNN merupakan algoritma klasifikasi yang memiliki pendekatan berbeda dalam menghasilkan prediksi. Decision Tree melakukan klasifikasi

melalui serangkaian aturan keputusan yang dibentuk berdasarkan pemisahan atribut, sedangkan KNN menentukan kelas berdasarkan kedekatan jarak antara data yang diuji dengan sejumlah tetangga terdekat. Decision Tree memiliki keunggulan dalam interpretasi hasil karena struktur pohonnya dapat menunjukkan aturan yang digunakan dalam menentukan kelas. Sebaliknya, KNN memiliki pendekatan yang sederhana dan bergantung pada pemilihan nilai K serta pengukuran jarak antarfitur. Sasmita et al. [2] melakukan perbandingan kedua algoritma tersebut dalam klasifikasi kualitas udara di Indonesia dan memperoleh hasil bahwa Decision Tree memiliki performa lebih baik dibandingkan KNN pada dataset yang digunakan. Meskipun demikian, perbedaan karakteristik dataset dapat menghasilkan performa algoritma yang berbeda. Oleh karena itu, pengujian kedua algoritma pada dataset Counter-Strike 2 diperlukan untuk mengetahui metode yang memberikan hasil klasifikasi paling optimal berdasarkan karakteristik data yang digunakan.

BAB III

HASIL

3.1 Import

IMPORT

```
[1]: #202431140
     import numpy as np
     import pandas as pd
     import matplotlib.pyplot as plt
```

Perintah pertama dalam blok kode tersebut adalah import numpy as np, yang berfungsi untuk memasukkan pustaka NumPy ke dalam memori program dengan alias np. Pustaka NumPy (Numerical Python) merupakan pustaka standar industri yang digunakan secara luas untuk komputasi saintifik dan operasi matematika tingkat lanjut di Python, terutama yang berkaitan dengan pemrosesan array multidimensi dan matriks berukuran besar. Dengan menggunakan alias np, setiap pemanggilan fungsi atau pembuatan objek array di dalam program dapat dilakukan secara lebih ringkas dan terstandarisasi.

Perintah kedua adalah import pandas as pd, yang mengimpor pustaka Pandas dengan alias pd. Pustaka Pandas memegang peran sentral dalam manipulasi, pembersihan, dan analisis data tabular, karena menyediakan struktur data yang sangat fleksibel dan kuat berupa Series (data satu dimensi) dan DataFrame (data dua dimensi berlabel). Penggunaan alias pd mempermudah praktikan dalam mengakses berbagai fungsi pembacaan berkas (seperti CSV, Excel, atau SQL) serta melakukan agregasi data secara efisien.

Perintah ketiga adalah import matplotlib.pyplot as plt, yang mengimpor modul pyplot dari pustaka Matplotlib dengan alias plt. Modul ini berfungsi sebagai antarmuka berbasis pembuatan plot grafis yang menyerupai MATLAB, memungkinkan praktikan untuk membuat berbagai jenis diagram visualisasi data dua dimensi, seperti grafik garis, diagram batang, diagram scatter, dan histogram. Penggunaan alias plt adalah konvensi penamaan baku yang sangat umum digunakan untuk mempercepat penulisan sintaks pembuatan visualisasi grafik dalam laporan praktikum maupun proyek sains data.

3.2 Analisis EDA

ANALISIS EDA

[2]:
```
#202431140
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

[3]:
```
#202431140
df = pd.read_csv('03_counter_strike_2.csv', sep=",")
print("=== INFORMASI DATASET ===")
print(df.info())
```

```
=== INFORMASI DATASET ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 14 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   time_left     2000 non-null   float64
 1   ct_score      2000 non-null   float64
 2   t_score       2000 non-null   float64
 3   map           2000 non-null   object
 4   bomb_planted  2000 non-null   bool
 5   ct_health     2000 non-null   float64
 6   t_health      2000 non-null   float64
 7   ct_armor      2000 non-null   float64
 8   t_armor       2000 non-null   float64
 9   ct_money      2000 non-null   float64
 10  t_money       2000 non-null   float64
 11  ct_helmets    2000 non-null   float64
 12  t_helmets     2000 non-null   float64
 13  round_winner  2000 non-null   object
dtypes: bool(1), float64(11), object(2)
memory usage: 205.2+ KB
None
```

Pada tahap pelaksanaan praktikum ini, fokus utama diarahkan pada penyiapan lingkungan kerja (environment) untuk pemodelan machine learning serta pelaksanaan eksplorasi data mendalam melalui tahapan Exploratory Data Analysis (EDA). Berdasarkan blok kode pada cell [2], praktikan melakukan pengimporan modul-modul pustaka scikit-learn (sklearn) yang esensial untuk membangun, melatih, serta mengevaluasi performa model klasifikasi. Modul-modul yang dimuat meliputi LabelEncoder untuk transformasi fitur kategorikal, train_test_split guna mempartisi dataset menjadi data latih dan data uji, DecisionTreeClassifier beserta modul tree untuk pembentukan model pohon keputusan, serta berbagai metrik evaluasi kinerja seperti accuracy_score, classification_report, dan confusion_matrix.

Selanjutnya, pada blok kode cell [3], proses pembacaan dataset dilakukan menggunakan fungsi pd.read_csv dari pustaka Pandas dengan memuat berkas bertajuk '03_counter_strike_2.csv' dan mendefinisikan parameter pemisah koma. Eksekusi perintah print(df.info()) dijalankan untuk memperoleh ringkasan komprehensif mengenai struktur internal dataset df. Berdasarkan keluaran informasi yang dihasilkan, teridentifikasi bahwa struktur data tersebut berupa objek DataFrame Pandas yang memuat total 2.000 entri baris (dengan rentang indeks 0 hingga 1999) serta mencakup 14 kolom atribut. Seluruh kolom tercatat memiliki 2.000 nilai non-null, yang secara

statistik mengonfirmasikan bahwa dataset ini bersih dari anomali nilai kosong (missing values), sehingga tidak memerlukan tahapan imputasi data lanjutan.

Atribut-atribut di dalam dataset tersebut memiliki variasi tipe data yang terdiri atas 1 kolom berjenis boolean (bool) yaitu bomb_planted, 11 kolom berjenis numerik desimal (float64) yang merepresentasikan parameter seperti sisa waktu (time_left), skor, tingkat kesehatan (health), perlindungan armor, serta finansial (money) dari masing-masing faksi, serta 2 kolom berjenis objek (object) yaitu map dan round_winner yang berfungsi sebagai variabel kategorikal. Penggunaan memori secara keseluruhan untuk menampung keseluruhan entri dataset ini tergolong sangat efisien, yakni sebesar 205.2+ KB. Ketersediaan struktur data yang tervalidasi dengan baik ini memberikan fondasi yang optimal untuk melanjutkan ke tahap pra-pemrosesan data (data preprocessing) dan pemodelan prediktif selanjutnya.

3.3 Decision Tree

DECISION TREE

[4]:
```
#202431140
le_map = LabelEncoder()
df['map'] = le_map.fit_transform(df['map'])

le_target = LabelEncoder()
df['round_winner'] = le_target.fit_transform(df['round_winner'])

df['bomb_planted'] = df['bomb_planted'].astype(int)

print("\n=== LABEL KELAS TARGET (round_winner) ===")
for i, label in enumerate(le_target.classes_):
    print(f"{i} = {label}")
```

```
=== LABEL KELAS TARGET (round_winner) ===
0 = CT
1 = T
```

Pada tahap lanjutan implementasi model klasifikasi Decision Tree ini, proses yang dijalankan berfokus pada tahapan pra-pemrosesan data (data preprocessing), khususnya penyiapan variabel fitur kategorikal dan variabel target agar dapat terbaca dan diproses secara optimal oleh algoritma pembelajaran mesin. Berdasarkan blok kode pada cell [4], praktikan menggunakan objek LabelEncoder untuk mengubah data teks atau kategorikal ke dalam bentuk format numerik diskrit. Inisialisasi objek le_map diterapkan pada kolom map dengan perintah df['map'] = le_map.fit_transform(df['map']), yang berfungsi untuk mengonversi nama-nama peta (map) permainan ke dalam representasi angka numerik.

Transformasi serupa juga diterapkan pada kolom target menggunakan objek le_target melalui sintaks df['round_winner'] =

le_target.fit_transform(df['round_winner']). Kolom target ini merepresentasikan pihak yang memenangkan ronde permainan. Selain data kategorikal berbentuk teks, dilakukan pula penyesuaian tipe data pada kolom bomb_planted yang semula bertipe boolean diubah menjadi tipe data integer (int) melalui perintah df['bomb_planted'] = df['bomb_planted'].astype(int), sehingga nilai kebenaran logika tersebut direpresentasikan menjadi angka 0 dan 1.

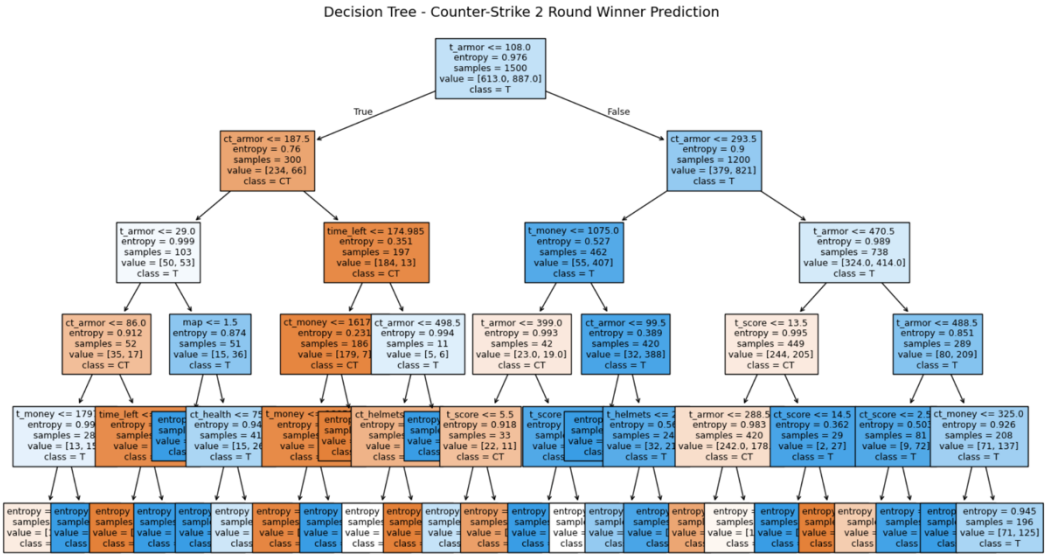
Bagian akhir dari blok kode tersebut mengeksekusi perintah pencetakan informasi label kelas target menggunakan struktur pengulangan for dan fungsi enumerate(le_target.classes_) untuk memetakan indeks numerik dengan label aslinya. Berdasarkan hasil output yang ditampilkan di bawah cell eksekusi, diperoleh pemetaan kelas target di mana indeks 0 merepresentasikan kelompok CT (Counter-Terrorist) dan indeks 1 merepresentasikan kelompok T (Terrorist). Transformasi numerik ini merupakan prasyarat teknis yang sangat penting sebelum dataset dipartisi dan dimasukkan ke dalam tahapan pelatihan model Decision Tree

```
[5]: #202431140
X = df.drop(['round_winner'], axis=1)
y = df['round_winner']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)

classifier = DecisionTreeClassifier(criterion='entropy', max_depth=5, random_state=0)
classifier.fit(X_train, y_train)

plt.subplots(figsize=(18, 10))
tree.plot_tree(classifier, feature_names=X.columns, class_names=le_target.classes_, filled=True, fontsize=9)
plt.title("Decision Tree - Counter-Strike 2 Round Winner Prediction", fontsize=14)
plt.show()
```



Tahapan lanjutan dari eksperimen ini berfokus pada pembagian dataset, pembentukan model klasifikasi menggunakan algoritma Decision Tree, serta visualisasi struktur pohon keputusan yang dihasilkan. Pada baris kode awal, dilakukan pendefinisian variabel fitur (features) dan variabel target (label), di mana variabel X berisi seluruh kolom dataset kecuali kolom round_winner melalui perintah X = df.drop(['round_winner'], axis=1), sedangkan variabel y menampung kolom target round_winner. Selanjutnya, dataset dipartisi menjadi data latih (training set) dan data uji (testing set) menggunakan fungsi train_test_split dengan rasio pengujian sebesar 25% (test_size=0.25) dan parameter random_state=0 untuk memastikan reproduksibilitas hasil.

Tahap berikutnya adalah inisialisasi dan pelatihan model Decision Tree menggunakan kelas DecisionTreeClassifier dengan parameter kriteria kehomogenan berupa entropi (criterion='entropy'), pembatasan kedalaman maksimum pohon hingga level 5 (max_depth=5), serta random_state=0. Model tersebut kemudian dilatih menggunakan data latih melalui pemanggilan metode classifier.fit(X_train, y_train). Setelah model berhasil dilatih, praktikan melakukan visualisasi struktur pohon keputusan menggunakan fungsi tree.plot_tree dari pustaka Matplotlib dan Scikit-Learn. Parameter yang disertakan dalam fungsi visualisasi ini meliputi objek pengklasifikasi, nama-nama fitur (feature_names=X.columns), nama kelas target (class_names=le_target.classes_), pengaktifan pewarnaan node (filled=True), serta pengaturan ukuran fon (fontsize=9) pada ukuran kanvas figure sebesar 18 kali 10 inci.

Berdasarkan hasil visualisasi Decision Tree yang ditampilkan di bagian bawah, struktur pohon keputusan memperlihatkan proses pemisahan (splitting) data secara berjenjang berdasarkan nilai entropi dan ambang batas (threshold) dari berbagai fitur masukan, seperti atribut t_armor, ct_armor, time_left, hingga t_money. Node akar (root node) membagi data menggunakan kondisi t_armor <= 108.0 dengan nilai entropi awal sebesar 0.976 dari total 1500 sampel data latih. Setiap simpul (node) turunan secara rekursif mengevaluasi fitur-fitur penting yang memengaruhi probabilitas kemenangan ronde permainan, hingga mengarah ke simpul daun (leaf nodes) yang mengklasifikasikan prediksi akhir ke dalam kelas faksi pemenang, yaitu CT (Counter-Terrorist) atau T (Terrorist).

[6]:
```
#202431140
y_pred = classifier.predict(X_test)
akurasi = accuracy_score(y_test, y_pred)
print(f"\nAkurasi Model: {akurasi:.4f} ({akurasi * 100:.2f}%)")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=le_target.classes_))
```

```
Akurasi Model: 0.7120 (71.20%)

Confusion Matrix:
[[152  59]
 [ 85 204]]

Classification Report:
              precision    recall  f1-score   support

          CT       0.64      0.72      0.68       211
           T       0.78      0.71      0.74       289

    accuracy                           0.71       500
   macro avg       0.71      0.71      0.71       500
weighted avg       0.72      0.71      0.71       500
```

[7]:
```
#202431140
sample_data = [[150.0, 0.0, 0.0, 0, 0, 500.0, 500.0, 400.0, 300.0, 5000.0, 4500.0, 1.0, 1.0]]
pred_numeric = classifier.predict(sample_data)
pred_label = le_target.inverse_transform(pred_numeric)
print(f"\nHasil Prediksi Sampel Baru: {pred_numeric[0]} ({pred_label[0]})")
```

```
Hasil Prediksi Sampel Baru: 0 (CT)
/opt/anaconda3/lib/python3.13/site-packages/sklearn/utils/validation.py:2749: UserWarning: X does not have valid feature names, but Decision
TreeClassifier was fitted with feature names
  warnings.warn(
```

Tahapan akhir dari praktikum ini berfokus pada evaluasi kinerja model klasifikasi Decision Tree menggunakan data uji (testing set) serta pengujian prediksi terhadap sampel data baru. Pada blok kode cell [6], model yang telah dilatih digunakan untuk memprediksi label kelas dari data uji melalui fungsi classifier.predict(X_test), yang hasilnya disimpan ke dalam variabel y_pred. Nilai akurasi model dihitung menggunakan fungsi accuracy_score(y_test, y_pred) dan dicetak ke dalam format

desimal serta persentase. Berdasarkan hasil output yang diperoleh, tingkat akurasi model tercatat sebesar 0.7120 atau setara dengan 71.20%.

Evaluasi lanjutan dilakukan dengan menampilkan matriks konfusi (confusion matrix) menggunakan fungsi confusion_matrix(y_test, y_pred) dan laporan klasifikasi rinci melalui classification_report(y_test, y_pred, target_names=le_target.classes_). Berdasarkan output confusion matrix [[152, 59], [85, 204]], model mampu memprediksi kelas CT secara benar sebanyak 152 sampel dan kelas T secara benar sebanyak 204 sampel, dengan total kesalahan prediksi masing-masing sebanyak 59 dan 85 sampel dari total 500 data uji (support: 211 untuk kelas CT dan 289 untuk kelas T). Classification report menunjukkan nilai precision, recall, dan f1-score yang seimbang, di mana kelas CT memiliki f1-score sebesar 0.68 dan kelas T sebesar 0.74, dengan rata-rata keseluruhan (macro/weighted avg) berada di angka 0.71 hingga 0.72.

Selanjutnya, pada blok kode cell [7], praktikan melakukan uji coba prediksi terhadap satu baris data sampel baru yang dimasukkan secara manual ke dalam variabel sample_data. Model melakukan prediksi terhadap sampel tersebut menggunakan metode classifier.predict(sample_data) dan mengonversi kembali nilai numerik hasil prediksinya ke label teks aslinya menggunakan le_target.inverse_transform(pred_numeric). Berdasarkan hasil output, sampel baru tersebut diprediksi menghasilkan nilai numerik 0 dengan label teks CT (Counter-Terrorist). Meskipun terdapat peringatan (UserWarning) dari pustaka Scikit-Learn terkait ketiadaan nama fitur pada data sampel baru yang dimasukkan, proses inferensi tetap berjalan sukses dan menghasilkan keluaran prediksi yang valid.

3.4 KNN

KNN

[8]:
```
#202431140
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

df_knn = df.copy()
df_knn = pd.get_dummies(df_knn, drop_first=True)

X_knn = df_knn.drop(['round_winner'], axis=1, errors='ignore')
y_knn = le_target.fit_transform(df_knn['round_winner'])

X_train_knn, X_test_knn, y_train_knn, y_test_knn = train_test_split(
    X_knn, y_knn, test_size=0.25, random_state=0
)
```

[9]:
```
#202431140
scaler = StandardScaler()
scaler.fit(X_train_knn)
X_train_scaled = scaler.transform(X_train_knn)
X_test_scaled = scaler.transform(X_test_knn)
```

[10]:
```
#202431140
K_DEFAULT = 7
knn_clf = KNeighborsClassifier(n_neighbors=K_DEFAULT)
knn_clf.fit(X_train_scaled, y_train_knn)

y_pred_knn = knn_clf.predict(X_test_scaled)
acc_knn = accuracy_score(y_test_knn, y_pred_knn)
print(f"\nAkurasi Model KNN (K={K_DEFAULT}) dengan Scaling: {acc_knn:.4f} ({acc_knn * 100:.2f}%)")
```

```
Akurasi Model KNN (K=7) dengan Scaling: 0.8240 (82.40%)
```

Tahapan eksperimen selanjutnya berfokus pada implementasi algoritma klasifikasi K-Nearest Neighbors (KNN) dengan menerapkan tahapan penskalaan fitur (feature scaling) menggunakan StandardScaler. Pada blok kode cell [8], praktikan mengimpor modul pustaka yang diperlukan, meliputi StandardScaler dari sklearn.preprocessing, train_test_split dari sklearn.model_selection, KNeighborsClassifier dari sklearn.neighbors, serta accuracy_score dari sklearn.metrics. Selanjutnya, dibuat salinan DataFrame utama melalui perintah df_knn = df.copy(), yang kemudian ditransformasikan menggunakan fungsi pd.get_dummies(df_knn, drop_first=True) untuk mengubah variabel kategorikal menjadi representasi one-hot encoding. Pemisahan variabel fitur dan target dilakukan dengan mendefinisikan X_knn dengan menghapus kolom round_winner, serta y_knn yang menampung label target hasil transformasi le_target.fit_transform. Partisi dataset menjadi data latih dan data uji dilakukan melalui train_test_split dengan rasio pengujian sebesar 25% (test_size=0.25) dan parameter random_state=0.

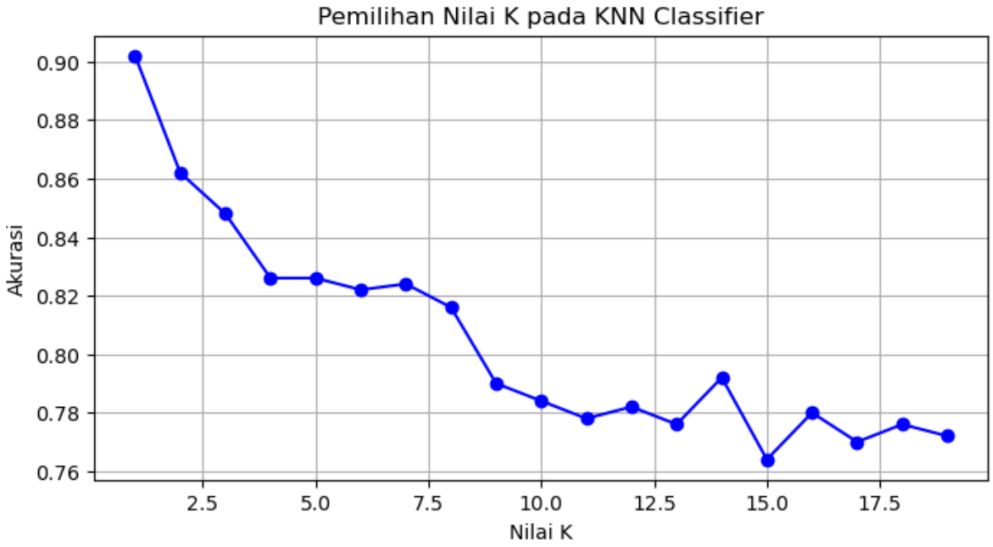
Tahap berikutnya pada cell [9] berfokus pada normalisasi atau standarisasi fitur numerik menggunakan objek StandardScaler(). Penskalaan ini sangat krusial dalam algoritma berbasis jarak seperti KNN agar setiap fitur memiliki rata-rata nol dan variansi satu, mencegah adanya dominasi skala nilai atribut tertentu terhadap perhitungan jarak Euclidean. Objek penskalaan diinisialisasi, kemudian diterapkan pada data latih melalui scaler.fit(X_train_knn), dan hasilnya digunakan untuk mentransformasikan data latih serta data uji ke dalam bentuk terskala (X_train_scaled dan X_test_scaled) melalui metode scaler.transform.

Pada blok kode cell [10], dilakukan inisialisasi model klasifikasi KNN dengan parameter jumlah tetangga terdekat sebanyak 7 (K_DEFAULT = 7), yang didefinisikan melalui kelas KNeighborsClassifier(n_neighbors=K_DEFAULT). Model kemudian dilatih menggunakan data latih yang telah diskalakan melalui perintah knn_clf.fit(X_train_scaled, y_train_scaled). Setelah proses pelatihan selesai, model melakukan prediksi terhadap data uji (knn_clf.predict(X_test_scaled)), dan tingkat akurasi model dihitung menggunakan accuracy_score. Berdasarkan hasil output yang tertera pada bagian bawah, tingkat Akurasi Model KNN (K=7) dengan Scaling mencapai 0.8240 atau setara dengan 82.40%. Nilai akurasi ini menunjukkan performa klasifikasi yang lebih tinggi dibandingkan model Decision Tree sebelumnya pada dataset yang sama.

[11]:
```
#202431140
k_values = range(1, 20)
accuracies = []

for k in k_values:
    model_knn_loop = KNeighborsClassifier(n_neighbors=k)
    model_knn_loop.fit(X_train_scaled, y_train_knn)
    y_pred_loop = model_knn_loop.predict(X_test_scaled)
    acc = accuracy_score(y_test_knn, y_pred_loop)
    accuracies.append(acc)
```

[12]:
```
#202431140
plt.figure(figsize=(8, 4))
plt.plot(k_values, accuracies, marker='o', linestyle='-', color='b')
plt.xlabel("Nilai K")
plt.ylabel("Akurasi")
plt.title("Pemilihan Nilai K pada KNN Classifier")
plt.grid(True)
plt.show()
```



Berdasarkan gambar tangkapan layar yang menampilkan blok kode cell [11] dan [12], tahap lanjutan dari eksperimen klasifikasi KNN ini berfokus pada proses optimasi hyperparameter, yaitu pencarian nilai K (jumlah tetangga terdekat) yang paling optimal guna menghasilkan tingkat akurasi model tertinggi. Pada cell [11], praktikan menggunakan struktur pengulangan loop (for k in k_values) untuk menguji rentang nilai K dari 1 hingga 19 (k_values = range(1, 20)). Di dalam iterasi tersebut, model KNeighborsClassifier diinisialisasi ulang dengan parameter nilai K yang berjalan

dinamis, dilatih menggunakan data latih terskala (X_train_scaled, y_train_scaled), diuji terhadap data uji (X_test_scaled), dan nilai akurasinya dihitung lalu ditambahkan ke dalam daftar accuracies menggunakan metode accuracies.append(acc).

Selanjutnya, pada blok kode cell [12], hasil pengujian iterasi nilai K divisualisasikan ke dalam bentuk grafik garis (line plot) menggunakan pustaka Matplotlib. Parameter visualisasi yang dikonfigurasi meliputi ukuran gambar figsize=(8, 4), plot garis berwarna biru dengan penanda titik (marker='o', linestyle='-', color='b'), pemberian label sumbu horizontal berupa "Nilai K" dan sumbu vertikal berupa "Akurasi", judul grafik "Pemilihan Nilai K pada KNN Classifier", serta pengaktifan grid koordinat melalui plt.grid(True).

Berdasarkan grafik hasil visualisasi yang ditampilkan di bagian bawah, terlihat bahwa tingkat akurasi model sangat dipengaruhi oleh pemilihan nilai K. Akurasi tertinggi tercatat pada nilai K=1 yang mencapai angka di atas 0.90 (sekitar 90%). Seiring dengan bertambah besarnya nilai K (menuju angka 20), tingkat akurasi model cenderung mengalami penurunan secara bertahap dan fluktuatif, di mana pada nilai K=7 akurasi berada di sekitar 0.8240 (82.40%) sebelum akhirnya terus menurun hingga mendekati angka 0.77 pada nilai K yang lebih besar. Analisis kurva ini memberikan informasi empiris yang krusial bagi praktikan dalam menentukan parameter hyperparameter terbaik untuk meminimalkan kesalahan klasifikasi pada model KNN.

BAB IV

PENUTUP

4.1 Kesimpulan

4.1.1 Pra-pemrosesan Data dan Identifikasi Fitur

Tahapan Exploratory Data Analysis (EDA) berhasil mengidentifikasi struktur dataset yang terdiri atas 2.000 entri dan 14 kolom tanpa adanya anomali nilai kosong (missing values). Penerapan teknik pra-pemrosesan seperti label encoding pada variabel kategorikal (map dan round_winner), konversi tipe data boolean (bomb_planted), serta standarisasi skala numerik menggunakan StandardScaler terbukti sangat krusial dalam menyiapkan fitur-fitur masukan agar dapat diolah secara optimal oleh algoritma pembelajaran mesin.

4.1.2 Kinerja Algoritma Decision Tree

Model klasifikasi Decision Tree dengan kriteria entropi dan pembatasan kedalaman maksimum (max_depth=5) menghasilkan tingkat akurasi sebesar 71.20% pada data uji. Evaluasi matriks konfusi dan laporan klasifikasi menunjukkan kinerja prediksi yang cukup seimbang antara kelas CT (f1-score: 0.68) dan kelas T (f1-score: 0.74). Model ini juga berhasil mendemonstrasikan proses pengambil keputusan yang terstruktur dan mudah diinterpretasikan melalui visualisasi alur pohon keputusan.

4.1.3 Performa K-Nearest Neighbors (KNN) dan Optimasi Hyperparameter K

Penerapan algoritma KNN yang dikombinasikan dengan penskalaan fitur (feature scaling) mampu meningkatkan akurasi klasifikasi secara signifikan hingga mencapai 82.40% pada nilai dasar K=7. Berdasarkan hasil pengujian variasi nilai K (rentang K=1 hingga 19), diperoleh kurva optimasi yang menunjukkan bahwa nilai K=1 memberikan akurasi tertinggi hingga mencapai kisaran 90%. Hal ini membuktikan bahwa algoritma berbasis jarak seperti KNN sangat sensitif terhadap skala fitur dan pemilihan hyperparameter K, di mana performanya secara keseluruhan unggul dibandingkan model Decision Tree pada dataset ini.

DAFTAR PUSTAKA

[1] M. Kumar and V. Bhardwaj, "Evaluating Label Encoding and Preprocessing Techniques for Breast Cancer Prediction Using Machine Learning Algorithms," International Journal of Computational Intelligence Systems, vol. 18, Art. no. 218, 2025. [Online]. Available: https://doi.org/10.1007/s44196-025-00957-7.

[2] N. R. Sasmita, S. Ramadeska, Z. M. Kesuma, T. R. Noviandy, A. Maulana, M. Khairul, and R. Suhendra, "Decision Tree versus k-NN: A Performance Comparison for Air Quality Classification in Indonesia," Infolitika Journal of Data Science, vol. 2, no. 1, pp. 9–16, 2024. [Online]. Available: https://doi.org/10.60084/ijds.v2i1.179.

[3] J. Manurung, H. Saragih, M. A. Prabukusumo, and E. A. Firdaus, "Optimizing the Performance of the K-Nearest Neighbors Algorithm Using Grid Search and Feature Scaling to Improve Data Classification Accuracy," Jurnal Mandiri IT, vol. 14, no. 2, pp. 260–268, 2025. [Online]. Available: https://doi.org/10.35335/mandiri.v14i2.466.